

Runtime-Side Security Profile Generation for OCI Containers

Ivan Platonov¹, Andrey Sadovykh², and Kirill Saltanov³

¹ Innopolis University, Innopolis, Russia
`i.platonov@innopolis.university`

² Softeam, Paris, France

`andrey.sadovykh@softeam.fr`

³ Innopolis University, Innopolis, Russia
`kirill.saltanov@innopolis.university`

Abstract. OCI containers share the host kernel, but practical confinement still depends on hand-written seccomp, AppArmor, and capability policies. This paper presents a security-oriented fork of `runc` that generates such policies inside the runtime. A normal `runc run -security-scan` invocation relaxes confinement only in memory, traces syscall and capability use with eBPF-based tools, learns AppArmor rules in complain mode, and writes a workload-specific profile set back to the OCI bundle. Later runs enforce those artefacts without carrying helper scripts in the bundle.

Evaluation combines an application-level command-injection laboratory, a kernel-boundary attack corpus, and a performance harness. Scan-then-enforce blocks the Flask laboratory attacks that succeed under stock `runc`. On a 22-reproducer corpus, the enforced configuration blocks all 17 applicable attack reproducers while allowing the positive control, including all 15 syscall, capability, filesystem, and lateral-movement probes; Docker-default upstream `runc` blocks 11/20 attempted cells and gVisor blocks 12/20. Performance measurements over 50 recorded repetitions show below 1% median overhead on CPU, memory, and loopback-network workloads, and about 5.5% on Redis, while gVisor is 30–73% slower than Docker on memory, network, and Redis.

Keywords: OCI runtime · `runc` · seccomp · AppArmor · Linux capabilities · eBPF.

1 Introduction

Containers are lightweight because they share a kernel, but that design also makes kernel-facing attack surface central to isolation [16, 3]. The OCI reference runtime, `runc`, is fast and widely deployed, yet strong confinement usually requires operators to supply correct seccomp, AppArmor, and capability profiles. Manual policy authoring is specialised work, and cluster-side recorders such as the Security Profiles Operator or Inspektor Gadget assume Kubernetes infrastructure and profile-management workflows [6, 4].

This paper asks whether the runtime can lower that operational cost. We implement a fork of `runc` that treats profile generation as a runtime-side learning phase. The contribution is threefold: (i) a single-flag workflow for generating syscall, capability, and AppArmor evidence; (ii) finalisation logic that rebuilds privileges from observed behaviour rather than merging with broad defaults; and (iii) self-contained scanner hooks implemented as hidden runtime sub-commands, preserving OCI bundle portability.

2 Runtime Design

The implementation extends `runc run` with an optional scanning branch. Without `-security-scan`, the bundle runs through the ordinary OCI path. With the flag, the runtime relaxes confinement in memory, installs tracing hooks, runs the container, stops tracers, and finalises generated profiles under the bundle’s `generated/` directory. The on-disk `config.json` is not widened before the scan.

Table 1. Scan artefacts.

Artefact	Role
<code>seccomp.json</code>	OCI seccomp allow-list from <code>oci-seccomp-bpf-hook</code> [2].
<code>capable-bpfcc.log</code>	Cgroup-wide capability trace consumed during finalisation [5].
<code>apparmor.profile</code>	AppArmor profile learned in complain mode and flipped to enforce mode.
<code>spec.original.json</code>	Backup of the pre-scan specification for auditability.

Capability finalisation normalises names to OCI `CAP_*` spelling and replaces all capability buckets from an empty set extended only with observed capabilities. This invariant prevents repeated scans from preserving privileges merely because they appeared in earlier configurations. AppArmor and seccomp are similarly evidence-based: the runtime records the exercised paths and syscalls, then embeds the resulting deny-by-default policies for subsequent runs.

3 Evaluation

All quantitative campaigns ran on `perf-bench`, an Ubuntu 24.04.4 VirtualBox guest with kernel 6.8.0-124, cgroup v2, four vCPUs (AMD Ryzen 7 7700), and AppArmor enabled for the security corpus. The VM lacks KVM and CPU-governor pinning; we therefore report medians and treat small differences as measurement noise.

3.1 Application Security Laboratory

The `runc-hardened-test` laboratory [12] packages a deliberately vulnerable Flask system-monitor application. A scan driven by legitimate UI, ping, and calendar-upload traffic produced a deny-by-default seccomp profile, an AppArmor profile, and a capability trace. After enforcement, the same command-injection payloads that succeeded under stock `runc` were blocked: the learned seccomp profile denied process-creation syscalls on the attack path, while AppArmor preserved legitimate application storage paths. A controlled ablation that re-enabled `execve` restored a shell, but most follow-on reads, mount probes, and persistence attempts remained blocked by the remaining layers.

3.2 Kernel-Boundary Attack Corpus

The `security-evaluation` harness [13] replays 22 shell-based reproducers across four configurations: C0 (Docker-default upstream `runc`), C1 (hardened `dpttk/runc` with a raw bundle), C2 (the same runtime with generated profiles enforced), and C3 (gVisor `runsc`). C0 is the weak security baseline. C3 is a sandbox reference, not the same mechanism as workload-specific profile enforcement.

An attack is blocked when it fails with policy or permission evidence, kernel audit denial, or absence of the expected side effect. Campaign 20260618-222423 stores `stdout/stderr` and `audit-window` logs for every attempted cell under `results/latest/logs`. Table 2 summarises the aggregate results.

Table 2. Security effectiveness: blocked/total by attack class. Misconfiguration rows apply only to Docker-backed configurations.

Class	C0	C1	C2	C3
Dangerous syscall	5/5	5/5	5/5	5/5
Capability-bound	3/4	4/4	4/4	3/4
Filesystem breakout	0/3	1/3	3/3	1/3
Post-RCE lateral	1/3	1/3	3/3	1/3
Misconfiguration	1/3	—	—	1/3
Runtime (scoped)	CVE	1/1	1/1	2/2
Positive control	0/1	0/1	0/1	0/1

C2 blocks all 15 syscall, capability, filesystem, and lateral-movement probes, compared with 9/15 for C0 and 10/15 for C3. The positive control succeeds in all configurations. Misconfiguration attacks such as `-privileged` and host PID namespace remain outside what bundle-level profiles can retrofit; treating them as explicit negative tests prevents overclaiming. CVE-2019-5736 is checked as a runtime-binary integrity guard rather than a destructive host-takeover replay [9];

CVE-2024-21626 FD-leak reproducers do not succeed on the tested runtime versions [10].

3.3 Performance Harness

The **performance-evaluation** harness [11] separates profile preparation from measurement. **stock** and **proposed** use the same hardened binary and OCI-bundle launcher; **stock** uses raw bundles, while **proposed** enforces generated seccomp, AppArmor, and capability profiles. Docker and gVisor are measured through Docker. This differs from C0 in the security corpus, where C0 means Docker-default upstream **runc**.

Campaign 20260616-193735 ran 10 warm-up and 50 recorded repetitions for sysbench CPU/memory, loopback iperf3, and Redis SET/GET. Throughput overhead is positive when the subject is slower than its baseline.

Table 3. Enforcement overhead: enforced profiles vs. raw bundles, same hardened binary and launcher.

Workload		Raw Enforced Overhead		
CPU (events/s)		2 332	2 346	−0.62%
Memory (MiB/s)		6 105 132	6 059 380	+0.75%
Network (Gbit/s)		55.20	55.85	−1.18%
Redis SET (req/s)		89 358	84 324	+5.63%
Redis GET (req/s)		89 542	84 587	+5.53%

Table 4. Sandbox overhead: gVisor vs. Docker-default baseline on the same host.

Workload	gVisor overhead
CPU	+2.57%
Memory	+73.45%
Network	+30.82%
Redis SET	+64.59%
Redis GET	+64.71%

Enforcement overhead is within noise for CPU, memory, and loopback network, and about 5.5% for Redis. gVisor provides a stronger isolation model but is substantially slower on memory, network, and Redis. On this VirtualBox host, Redis AppArmor loading failed with **apparmor/exec**; the Redis enforcement row therefore reflects seccomp and capability enforcement primarily.

4 Discussion

Runtime-side profiling is a middle ground between default `runc` and full sandboxing. It does not replace gVisor’s kernel-surface reduction, but it narrows a shared-kernel runtime with low measured overhead and without requiring a cluster recorder. Its main safety invariant is replace-from-evidence: finalisation does not union observed privileges with historical broad defaults.

The approach has clear limits. Dynamic profiles cover only exercised behaviour; rare paths need representative tests. UID 0 workloads can under-report capability checks. File capabilities embedded in executable xattrs, such as the raw-network privilege on `ping`, are not yet merged automatically. Finally, deployment mistakes such as `-privileged` containers or host namespace sharing bypass what bundle-level scan output can repair.

5 Related Work

Prior work compares native `runc`, gVisor, and Kata in terms of performance and isolation [16, 14, 15], but rarely evaluates `runc` with workload-specific generated policies. Lopes et al. [7] automate seccomp profile generation; Mattetti and Allouche [8] argue for layered Linux hardening. Cluster-side recorders such as the Security Profiles Operator, Inspektor Gadget, and Tetragon target Kubernetes environments [6, 4, 1]. Our work moves the same evidence-collection idea into the OCI runtime layer.

6 Conclusion

This paper presented a `runc` fork that generates and enforces workload-specific seccomp, AppArmor, and capability profiles. The runtime blocks command-injection exploitation in an application laboratory and blocks all 15 kernel-boundary probes in a reproducible attack corpus, while Docker-default upstream `runc` and gVisor allow several filesystem, lateral-movement, or misconfiguration cases in the same matrix. Enforcement adds near-zero overhead on CPU, memory, and loopback-network workloads and about 5.5% on Redis, well below gVisor’s sandbox overhead on the same host. Within its stated limits, runtime-side profiling offers a practical way to raise the security floor for plain OCI deployments.

Acknowledgments. Artefacts are published at <https://github.com/dpttk>.

Disclosure of Interests. The authors have no competing interests to declare.

References

1. Cilium / Isovalent: Tetragon: eBPF-based security observability and runtime enforcement. <https://github.com/cilium/tetragon> (2025), accessed 2026-04-19

2. containers organisation: oci-seccomp-bpf-hook: Oci prestart hook that traces syscalls and emits a seccomp profile. <https://github.com/containers/oci-seccomp-bpf-hook> (2024), accessed 2026-04-19
3. Flauzac, O., Gonzalez, C., Nolot, F.: A review of native container security for running applications. *International Journal of Computer Science and Engineering* **22**(3), 187–204 (2020)
4. Inspektor Gadget contributors: Inspektor gadget: collection of ebpf-based tools for inspecting and debugging kubernetes workloads. <https://github.com/inspektor-gadget/inspektor-gadget> (2025), accessed 2026-04-19
5. IO Visor / iovisor: Bcc: Tools for bpf-based linux io analysis, networking, monitoring, and more (capable, capable-bpfcc). <https://github.com/iovisor/bcc> (2025), accessed 2026-04-19
6. Kubernetes SIG Auth, kubernetes-sigs: Security profiles operator. <https://github.com/kubernetes-sigs/security-profiles-operator> (2025), accessed 2026-04-19
7. Lopes, N., Martins, R., Correia, M.E., Serrano, S., Nunes, F.: Container hardening through automated seccomp profiling. In: *Proceedings of the 6th International Workshop on Container Technologies and Container Clouds*. pp. 31–36. ACM (2020). <https://doi.org/10.1145/3429885.3429966>
8. Mattetti, M., Allouche, Y.: Automatic security hardening and protection of linux containers. *Security and Privacy in Communication Systems* (2020)
9. National Vulnerability Database: Cve-2019-5736: runc container escape vulnerability (2019)
10. National Vulnerability Database: Cve-2024-21626: runc process cwd and leaked file descriptors (2024)
11. Platonov, I.: performance-testing: Oci runtime performance evaluation harness. <https://github.com/dpttk/performance-testing> (2026), accessed 2026-06-17
12. Platonov, I.: runc-hardened-test: security laboratory for scan-then-enforce evaluation. <https://github.com/dpttk/runc-hardened-test> (2026), accessed 2026-06-17
13. Platonov, I.: security-evaluation: reproducible attack corpus and security matrix harness. <https://github.com/dpttk/security-evaluation> (2026), accessed 2026-06-18
14. Viktorsson, W., Klein, C., Tordsson, J.: Security-performance trade-offs of kubernetes container runtimes. In: *2020 IEEE MASCOTS*. pp. 1–4. IEEE (2020)
15. Volpert, S., Winkelhofer, S., Wesner, S., Domaschka, J.: An empirical analysis of common oci runtimes' performance isolation capabilities. In: *Proceedings of the 15th ACM/SPEC International Conference on Performance Engineering*. pp. 60–70. ACM (2024). <https://doi.org/10.1145/3629526.3645044>
16. Wang, X., Johannesson, D.: Performance and isolation analysis of runc, gvisor and kata containers runtimes. *Cluster Computing* **25**, 2363–2380 (2022). <https://doi.org/10.1007/s10586-021-03517-8>